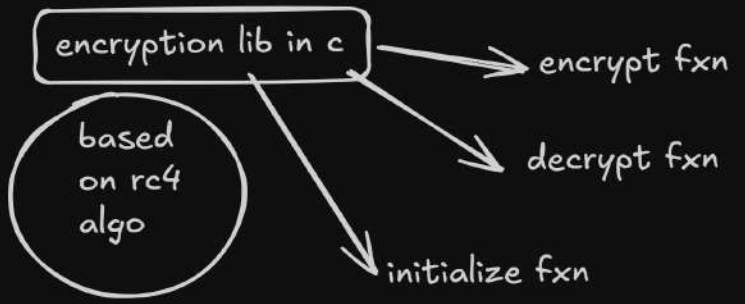
phase 1 - declaring datatypes and basic fuctions. seting up example.c and lib.h files.



int8*: A pointer to an 8-bit integer (often used for strings or raw byte arrays/buffers)

.int16*:pointer to A standard 16-bit integer (often used for length, sizes, or keys).

1 . IN EXAMPLE-
- (a). define pointers - key , from , encryped , decrypted text.
- (b) . defining a pointer that points to the state of our engine in
- (c) . defining size/length var. for key and text
- (d). using lib by initializing with a key and size(key)
  [use - init. fxn]
- (e.) messages and printing of 'from'
- (f.) encryption fxn calling with text and its size.
- (g.) fxn to output the encrypted text. [a binary printer]

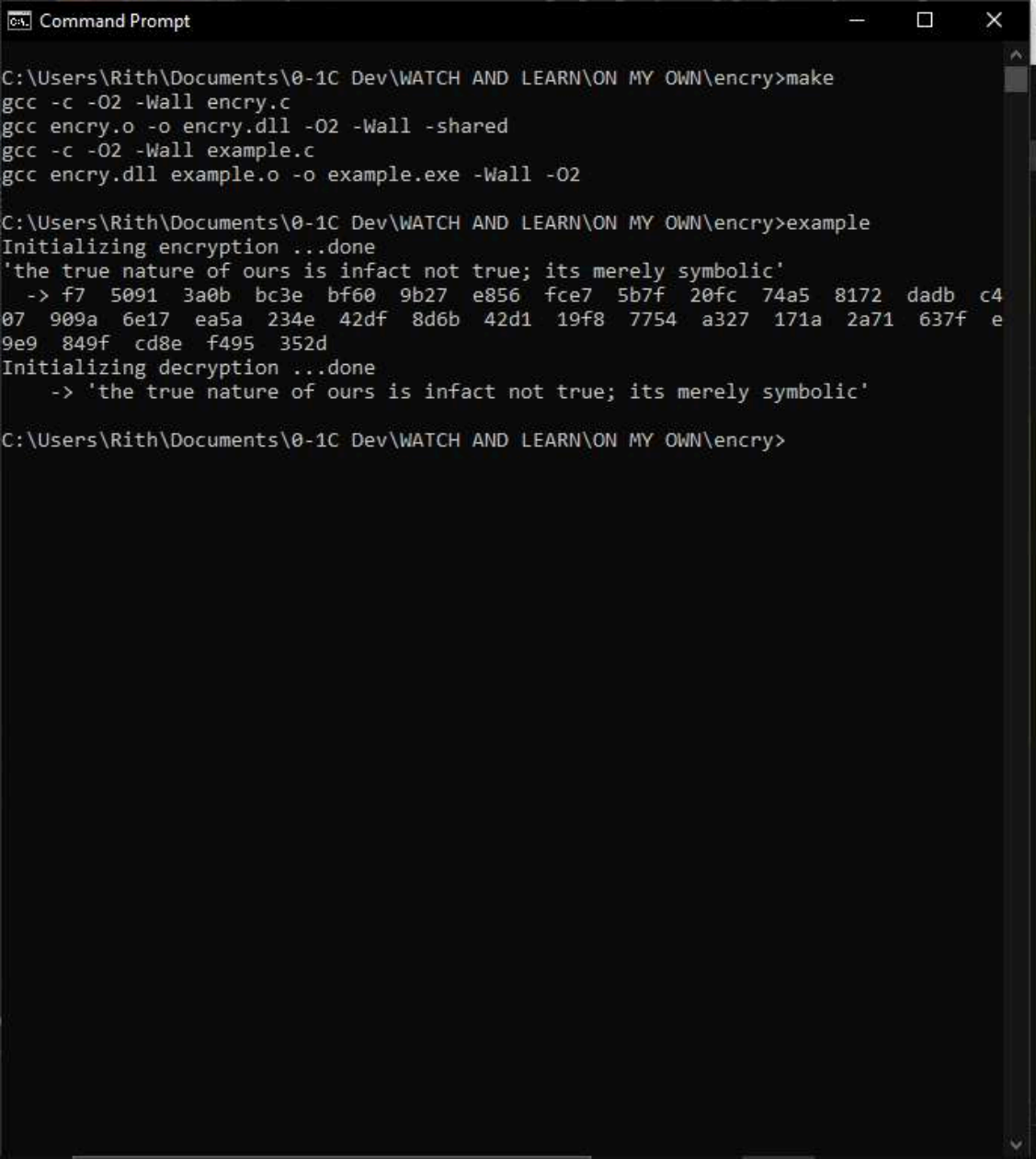
encrypt.h
1.main fxn
#we are using pointer to directly point at the adress of the variables and data needed/given by user

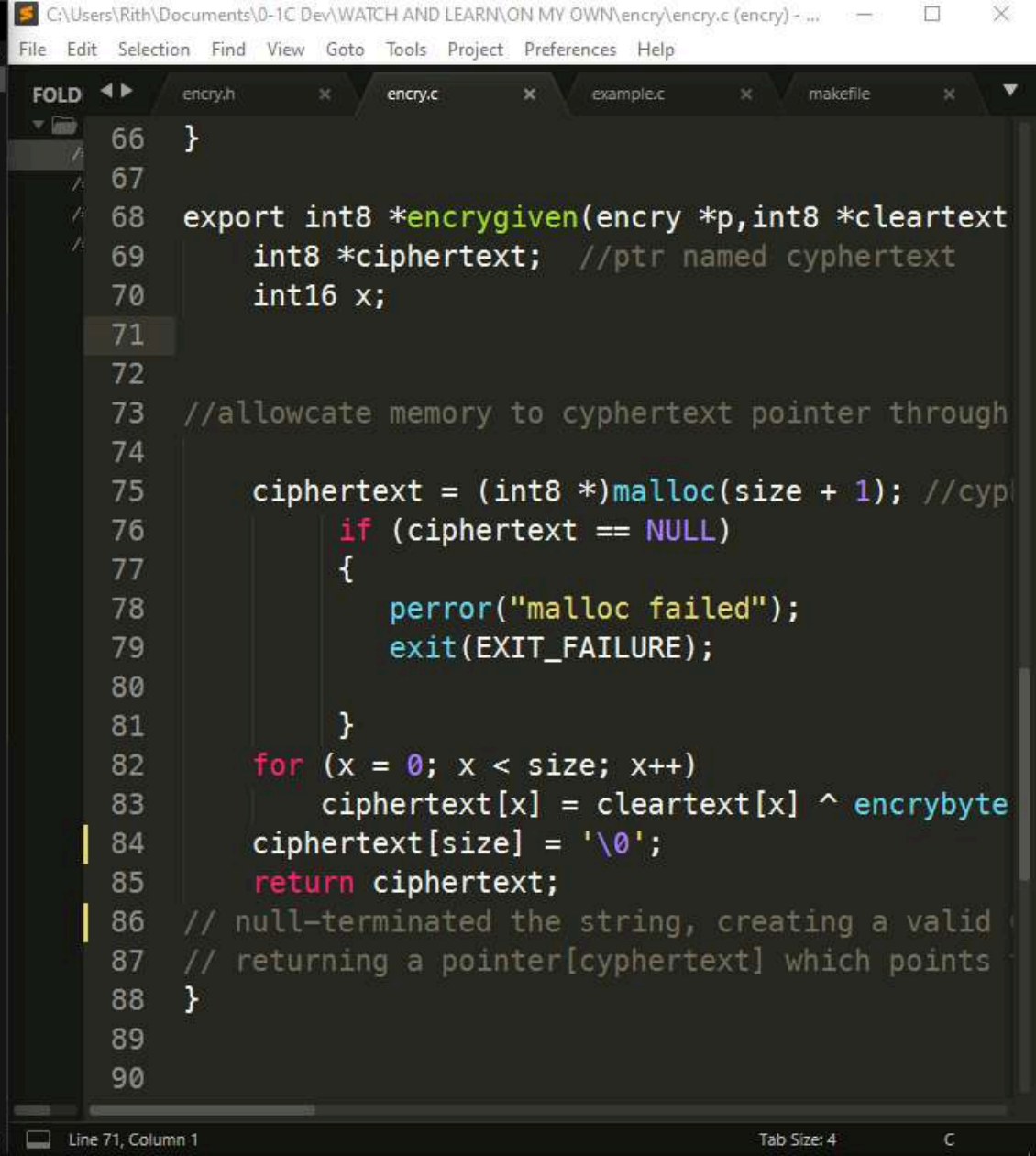
2.using 8-char , 16bit - int, 32bit-int - define them by typedef as [8-32bit]key is allowed!

2.defining a structure to group related var. of diff d-t we will use.

3. function to initialize rc4 which takes a key and text to encrypt or decrypt it.
#check always if fxn returns 0 , otherwise no code for null will crash it , malloc will return error..

4.when init fxn used - will give the defined strucure back
5. encrypt(text , size of text) --> pointer to encryped text
6.fxn to decrypt - just call encryption fxn.
7. a fxn to return encrpted text single char.


Command Prompt
C:\Users\Rith\Documents\0-1C Dev\WATCH AND LEARN\ON MY OWN\encry>make
gcc -c -O2 -Wall encry.c
gcc encry.o -o encry.dll -O2 -Wall -shared
gcc -c -O2 -Wall example.c
gcc encry.dll example.o -o example.exe -Wall -O2
C:\Users\Rith\Documents\0-1C Dev\WATCH AND LEARN\ON MY OWN\encry>example
Initializing encryption ...done
'the true nature of ours is infact not true; its merely symbolic'
-> f7 5091 3a0b bc3e bf60 9b27 e856 fce7 5b7f 20fc 74a5 8172 dadb c4
07 909a 6e17 ea5a 234e 42df 8d6b 42d1 19f8 7754 a327 171a 2a71 637f e
9e9 849f cd8e f495 352d
Initializing decryption ...done
-> 'the true nature of ours is infact not true; its merely symbolic'
C:\Users\Rith\Documents\0-1C Dev\WATCH AND LEARN\ON MY OWN\encry>


C:\Users\Rith\Documents\0-1C Dev\WATCH AND LEARN\ON MY OWN\encry\encry.c (encry) - ...
File Edit Selection Find View Goto Tools Project Preferences Help
FOLD
encry.h
encry.c
example.c
makefile
66 }
67
68 export int8 *encrygiven(encry *p,int8 *cleartext
69 int8 *ciphertext; //ptr named cyphertext
70 int16 x;
71
72
73 //allowcate memory to cyphertext pointer through
74
75 ciphertext = (int8 *)malloc(size + 1); //cyp
76 if (ciphertext == NULL)
77 {
78 perror("malloc failed");
79 exit(EXIT_FAILURE);
80
81 }
82 for (x = 0; x < size; x++)
83 ciphertext[x] = cleartext[x] ^ encrybyte
84 ciphertext[size] = '\0';
85 return ciphertext;
86 // null-terminated the string, creating a valid
87 // returning a pointer[cyphertext] which points
88 }
89
90
Line 71, Column 1
Tab Size: 4
C

pointers and datatypes flow -

1. created(typedefined) custom datatypes from primitive dt

typedef . modification . prmitv dt . name of your dt

typedef unisgned char int8

meaning : int8 is a custom dt — made up of → unsigned char

my dt	primt dt	used for
int8	char + unsigned	key / text
int16	short+unsigned	sizes/bits
int32		

. char =Array[characters] -> String

ex: char = ashi -> ['a' , 's' , 'h' , 'i']

·type-conversions-

convert key into a int8 dt and a pointer

(int8 *)key

function datatypes -

output of certain dt which a fxn will return.

define-

data type . fxn-name (dt of input1 , dt of input2)

int8 encrypt (int8* , int16)

means: the encrypt fxn will take ptr of an int8 dt variable and int16 dt variable and return a int8 dt output

fxn can be of=return a -> structure , custom dt , primtv dt

structure for lib: define a structue for lib to use when it is initialized. give us required data to perform operations..

use structure as a dt

1. define it → struct X { //define your collection of data types here };

2. typedef it → typedef struct X nameof_custom_structure_dt

rc4 algorithm - used for encryption. [see wiki]

initializing the crypto → produce one encrypted byte

take key and prepare the internal state
of crypto engine

used - loop iteration over the size of key

- int8 ; . need: i and j and a key
  array - S[256 of 8 bit values]

write code for rc4 algo init in
initialization fxn of lib.h
[for loops , tmp var , geting i,j,k ,
and array from structure through
pointer. returns a ptr containing struct.

always use ptr. when acessing
values of var from a struct !
to use in code ...

STRUCTURE - things[dt key] we get when initialize the engine from user

p->x : to access a variable-s value inside
a structure (struct) when you are
using a pointer.

take out the value of x defined in struct.
when pointer is pointing towards its address.

- int8 -> 255 max {unsigned}

^ is XOR
if you have => it will give
0 0 => 1
1 0 => 1
0 1 => 1 # ^ used with the keystream
1 1 => 0 like streamcypher prod by rc4

bytefxn - write rc4 algorithm ! | takes bytefxn(struct)---> k [one byte decoded] run until every byte is decoded.

Encryption fxn- write rc4 algo ! | encryptionfxn(struct pointer , int8.ptr , int18)

{cleartext = int8 = unsigned char = char array.
cyphertext= int8= unsigned char = char array.}

ptr [text need to be encryted]

[size of the text to be encrypted]

keystream = a random or pseudorandom sequence of data combined with a plaintext message to create an encrypted ciphertext

ciphertext[x] = cleartext[x] ^ encrybyte(p);

array of char x->256 stores every digit of encrypted text

array of char x->256 stores every digit of unenc or clear text

XOR WITH

fxn. that returns 1 byte of keystream

int8 datatype
- unsigned char
- array of char
- convert 8bit binary
- 1 byte of data
- 1 byte stored at every index of the array

int8 datatype
- unsigned char
- array of char
- convert 8bit binary
- 1 byte of data
- 1 byte stored at every index of the array

to produce one byte of the keystream
prga - runs a loop
1 iteration -> 1 byte
uses int8 ptrs - i , j

1 BYTE OF CYPHER TEXT <- is prod by - 1 BYTE OF CLEAR TEXT XOR WITH 1 BYTE OF KEYSTREAM

encryption fxn(cleartext , size) --> cipher text

int8 int16 int8

` size allowcation for cypher text.

int8 - store value in 1 byte; in its range similarly int16 , int32 also !

int8/int16/int32 => computers store these integers internally using binary representation (zeros and ones) in hardware,

[Secret Key]

1. KEY SCHEDULING (KSA) ——▶ Creates a shuffled, random 256-byte Array (S)

2. KEYSTREAM GENERATION (PRGA)
   - ▶ Move index pointers (i and j)
   - ▶ Swap numbers in Array S
   - ▶ Math Lookup ——▶ Generates 1 Keystream Byte

3. ENCRYPTION ——▶ [Keystream Byte]

⊕ (XOR)
[Plaintext Byte]

[Ciphertext Byte]

DECYPT- must uninit the encryption engine , then we can call the decryption fxn

the decrypt fxn will call the encrypt fxn which when given an already encrypted text will return a decryped one!

uninit fxn : free up the memory

#define uninit_fxn(x) free(x)